

# **Test Plan Document**

**for**

# **Restaurant Management System**

| Test Case ID | Name of Module           | Test case description                                                                                             | Pre-conditions                                                                        | Test Steps                                                                                                                                                                                       | Test data                                                                       | Expected Results                                                                                             | Actual Result | Test Result |
|--------------|--------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------|-------------|
| UT_01        | Table Booking Management | Verify the system displays available tables for a valid date, time and party size (REQ-1)                         | Booking service is running; at least one table exists in the database                 | 1: Call the availability-check function with a date, time slot and party size<br>2: Inspect the list of tables returned                                                                          | Date: 2026-09-20, Time: 19:00, Party size: 4                                    | Function returns all tables with capacity >= 4 that have no existing booking overlapping 19:00 on 2026-09-20 |               |             |
| UT_02        | Table Booking Management | Verify the system returns no tables when none are free for the requested slot (REQ-1, negative)                   | All tables are already booked for the requested slot                                  | 1: Call the availability-check function for a fully-booked date/time/party size<br>2: Inspect the returned list                                                                                  | Date: 2026-09-20, Time: 19:00, Party size: 2 (slot fully booked)                | Function returns an empty list and the system suggests the closest available slot                            |               |             |
| UT_03        | Table Booking Management | Verify creating a booking locks the selected table for its reserved slot (REQ-2)                                  | Table 5 is free for the target slot                                                   | 1: Submit a booking request for Table 5 at a given date/time<br>2: Query the table's status immediately after                                                                                    | Table: 5, Date: 2026-09-20, Time: 19:00, Party size: 2                          | Booking record is created and Table 5's status is 'Reserved' for that slot                                   |               |             |
| UT_04        | Table Booking Management | Verify a booking cannot be created for a table already locked in an overlapping slot (REQ-2, negative)            | Table 5 already has a confirmed booking for 19:00-20:30                               | 1: Submit a second booking request for Table 5 at 19:30 on the same date<br>2: Observe the system response                                                                                       | Table: 5, Date: 2026-09-20, Time: 19:30 (overlaps existing 19:00-20:30 booking) | Booking request is rejected with a message indicating the table is unavailable for that slot                 |               |             |
| UT_05        | Table Booking Management | Verify a booking can be cancelled more than 30 minutes before the reserved time (REQ-3)                           | A confirmed booking exists for a slot more than 30 minutes away                       | 1: Submit a cancellation request for the booking<br>2: Query the booking status and table status                                                                                                 | Booking reserved for 19:00; cancellation submitted at 18:00                     | Booking status changes to 'Cancelled' and the table is released back into the available pool                 |               |             |
| IT_01        | Table Booking Management | Verify the booking API persists a new booking to the database and updates table status (REQ-1, REQ-2)             | Booking API and database are both running and reachable                               | 1: Send a POST request to the booking endpoint with valid date/time/party size/table<br>2: Query the database directly for the new booking record<br>3: Query the table's status in the database | POST /api/bookings {table:5, date:'2026-09-20', time:'19:00', guests:2}         | API returns 201 Created; a matching row exists in the Bookings table; Table 5's status is 'Reserved'         |               |             |
| IT_02        | Table Booking Management | Verify the staff dashboard retrieves and displays current bookings from the booking service (REQ-4)               | At least 3 bookings exist for the current day; staff user is logged in                | 1: Log in as a Staff user<br>2: Open the Bookings section of the admin dashboard<br>3: Compare the list shown against the database records                                                       | Staff credentials; 3 seeded bookings for today                                  | All 3 bookings are listed on the dashboard with correct table, time and customer details                     |               |             |
| IT_03        | Table Booking Management | Verify staff can manually cancel a booking from the dashboard and the change reflects in the database (REQ-4)     | A confirmed booking exists; staff user is logged in                                   | 1: Log in as Staff<br>2: Open the booking in the dashboard and select 'Cancel Booking'<br>3: Query the database for the booking's status                                                         | Existing booking ID: BK-1001                                                    | Dashboard shows the booking as 'Cancelled'; database record for BK-1001 has status 'Cancelled'               |               |             |
| ST_01        | Table Booking Management | End-to-end: customer books a table via the UI and receives a confirmation with a booking reference (REQ-1, REQ-2) | Customer is on the Table Booking page; at least one table is free for the chosen slot | 1: Select a date, time and party size<br>2: Choose a table from the available options and confirm<br>3: Observe the confirmation screen                                                          | Date: 2026-09-21, Time: 20:00, Party size: 3                                    | Booking is confirmed and a unique booking reference number is displayed to the customer                      |               |             |
| ST_02        | Table Booking Management | End-to-end: customer attempts to double-book an already reserved slot; system rejects and                         | The chosen table/slot combination is                                                  | 1: Select a date, time and party size that is already fully booked<br>2: Attempt to confirm the booking<br>3: Observe the system's response                                                      | Date: 2026-09-21, Time: 20:00 (already fully                                    | Booking is rejected with a message, and the system suggests the closest available time slot                  |               |             |

| Test Case ID | Name of Module  | Test case description                                                                                                 | Pre-conditions                                                           | Test Steps                                                                                                                                                      | Test data                                                               | Expected Results                                                                                         | Actual Result | Test Result |
|--------------|-----------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------|-------------|
|              |                 | suggests the nearest available slot (REQ-1, REQ-2)                                                                    | already fully booked                                                     |                                                                                                                                                                 | booked), Party size: 3                                                  |                                                                                                          |               |             |
| UT_06        | Menu Management | Verify an admin can create a new menu item with valid name, price and category (REQ-5)                                | Admin is authenticated                                                   | 1: Submit a create-menu-item request with valid name, price, category and availability<br>2: Query the menu items table for the new entry                       | Name: 'Paneer Tikka', Price: 220, Category: 'Starters', Available: true | Menu item is created successfully and appears in the menu items table with the given values              |               |             |
| UT_07        | Menu Management | Verify menu item creation is rejected when the price is negative or a required field is missing (REQ-5, negative)     | Admin is authenticated                                                   | 1: Submit a create-menu-item request with a negative price and a missing name<br>2: Observe the system response                                                 | Name: ", Price: -50, Category: 'Starters'                               | Request is rejected with a validation error identifying the invalid/missing fields; no record is created |               |             |
| UT_08        | Menu Management | Verify an admin can update an existing menu item's details (REQ-5)                                                    | Menu item 'Paneer Tikka' already exists                                  | 1: Submit an update request changing the item's price and availability<br>2: Query the menu items table for the updated entry                                   | Item: 'Paneer Tikka', New price: 240, Available: false                  | Menu item record is updated with the new price and availability status                                   |               |             |
| UT_09        | Menu Management | Verify an admin can delete a menu item (REQ-5)                                                                        | Menu item 'Paneer Tikka' exists and is not referenced by an open order   | 1: Submit a delete request for the menu item<br>2: Query the menu items table                                                                                   | Item: 'Paneer Tikka'                                                    | Menu item is removed from the menu items table and no longer returned by menu queries                    |               |             |
| UT_10        | Menu Management | Verify only menu items marked 'available' are returned by the menu listing function (REQ-6)                           | Menu contains a mix of available and unavailable items                   | 1: Call the menu-listing function<br>2: Inspect the returned list of items                                                                                      | 5 items marked available, 2 items marked unavailable                    | Only the 5 available items are returned; the 2 unavailable items are excluded                            |               |             |
| UT_11        | Menu Management | Verify menu items are grouped correctly by category (REQ-7)                                                           | Menu contains items across Starters, Main Course and Desserts categories | 1: Call the menu-listing function with grouping enabled<br>2: Inspect the grouped output                                                                        | 3 Starters, 4 Main Course, 2 Desserts items                             | Items are grouped under their correct category headings with correct counts in each group                |               |             |
| IT_04        | Menu Management | Verify a menu update made via the admin panel is immediately reflected in the customer-facing menu API (REQ-5, REQ-6) | Admin is logged in; customer-facing menu API is reachable                | 1: Update an existing item's price via the admin panel<br>2: Call the customer-facing menu API for the same item                                                | Item: 'Paneer Tikka', updated price: 250                                | Customer-facing API returns the item with the updated price of 250, with no manual refresh needed        |               |             |
| IT_05        | Menu Management | Verify marking an item unavailable removes it from the customer menu listing returned by the API (REQ-6)              | Item is currently available and visible to customers                     | 1: Mark the item as unavailable via the admin panel<br>2: Call the customer-facing menu listing API                                                             | Item: 'Paneer Tikka' marked unavailable                                 | The item no longer appears in the customer-facing menu API response                                      |               |             |
| ST_03        | Menu Management | End-to-end: admin adds a new Starters item; customer browsing the menu sees it under Starters (REQ-5, REQ-7)          | Admin is logged in; customer has the menu page open                      | 1: Admin adds a new item 'Spring Rolls' under category Starters<br>2: Customer refreshes/browses the menu page<br>3: Customer looks under the Starters category | Item: 'Spring Rolls', Category: 'Starters', Price: 180                  | 'Spring Rolls' appears under the Starters category on the customer-facing menu                           |               |             |
| ST_04        | Menu Management | End-to-end: admin marks an item unavailable; customer no longer sees it                                               | Item is currently visible to the customer                                | 1: Admin marks the item as unavailable<br>2: Customer browses/refreshes the menu page                                                                           | Item: 'Spring Rolls' marked unavailable                                 | 'Spring Rolls' no longer appears on the customer-facing menu                                             |               |             |

| Test Case ID | Name of Module                   | Test case description                                                                                                            | Pre-conditions                                                             | Test Steps                                                                                                                                            | Test data                                                      | Expected Results                                                                                          | Actual Result | Test Result |
|--------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|---------------|-------------|
|              |                                  | while browsing the live menu (REQ-6)                                                                                             |                                                                            |                                                                                                                                                       |                                                                |                                                                                                           |               |             |
| UT_12        | Food Ordering and Order Tracking | Verify a customer can add one or more available menu items to the cart (REQ-8)                                                   | At least 2 menu items are marked available                                 | 1: Add 'Paneer Tikka' to the cart<br>2: Add 'Spring Rolls' to the cart<br>3: Inspect the cart contents                                                | Items: Paneer Tikka (x1), Spring Rolls (x2)                    | Cart contains both items with the correct quantities                                                      |               |             |
| UT_13        | Food Ordering and Order Tracking | Verify submitting a cart creates a single order with status 'Received' (REQ-8, REQ-9)                                            | Cart contains at least one available item                                  | 1: Submit the cart as an order<br>2: Query the orders table for the new record                                                                        | Cart: Paneer Tikka (x1), Spring Rolls (x2)                     | A single order record is created containing both items with status 'Received'                             |               |             |
| UT_14        | Food Ordering and Order Tracking | Verify staff can update an order's status from Received to Preparing to Served (REQ-9)                                           | An order exists with status 'Received'                                     | 1: Staff updates the order status to 'Preparing'<br>2: Staff updates the order status to 'Served'<br>3: Query the order's status after each update    | Order ID: OR-2001                                              | Order status changes to 'Preparing' and then to 'Served' as each update is applied                        |               |             |
| UT_15        | Food Ordering and Order Tracking | Verify an order is rejected if it contains a menu item marked unavailable, with correct notification (REQ-11, negative)          | One item in the cart has been marked unavailable after being added         | 1: Add an item to the cart while it is available<br>2: Admin marks the item unavailable<br>3: Submit the cart as an order                             | Cart contains 'Spring Rolls', which is then marked unavailable | Order submission fails; customer is notified that 'Spring Rolls' is no longer available and was removed   |               |             |
| UT_16        | Food Ordering and Order Tracking | Verify the cart correctly calculates the total price for multiple items and quantities (REQ-8)                                   | Menu item prices are known                                                 | 1: Add Paneer Tikka (price 220) x1 and Spring Rolls (price 180) x2 to the cart<br>2: Inspect the cart's calculated total                              | Paneer Tikka x1 (220), Spring Rolls x2 (180 each)              | Cart total is correctly calculated as $220 + (180 \times 2) = 580$                                        |               |             |
| IT_06        | Food Ordering and Order Tracking | Verify an order-status change in the backend triggers a WebSocket push event to the Node.js real-time service (REQ-10)           | Node.js WebSocket service is running and connected to the Django backend   | 1: Update an order's status via the staff dashboard<br>2: Inspect the WebSocket event emitted by the Node.js service                                  | Order ID: OR-2001, new status: 'Preparing'                     | A WebSocket event is emitted containing the order ID and the new status 'Preparing'                       |               |             |
| IT_07        | Food Ordering and Order Tracking | Verify the customer's order-tracking screen updates in real time upon receiving a WebSocket status push (REQ-10)                 | Customer's order-tracking page is open with an active WebSocket connection | 1: Trigger an order-status update on the backend<br>2: Observe the customer's order-tracking screen without refreshing                                | Order ID: OR-2001, status change: 'Received' -> 'Preparing'    | The order-tracking screen updates to show 'Preparing' within 1 second, with no manual refresh             |               |             |
| IT_08        | Food Ordering and Order Tracking | Verify order submission correctly persists the order and order items in the database with correct totals (REQ-8)                 | Cart contains valid available items                                        | 1: Submit the cart as an order<br>2: Query the Orders and OrderItems tables                                                                           | Cart: Paneer Tikka x1, Spring Rolls x2                         | Orders table has one new row; OrderItems table has two rows linked to it; total amount matches cart total |               |             |
| ST_05        | Food Ordering and Order Tracking | End-to-end: customer places an order, staff updates status through all stages, customer sees live updates (REQ-8, REQ-9, REQ-10) | Customer has items in the cart; staff dashboard is open                    | 1: Customer submits the order<br>2: Staff updates status to 'Preparing', then 'Served'<br>3: Customer observes their order-tracking screen throughout | Cart: Paneer Tikka x1, Spring Rolls x2                         | Customer's screen updates live through Received -> Preparing -> Served without any page refresh           |               |             |
| ST_06        | Food Ordering and Order Tracking | End-to-end: customer attempts to order an item that becomes unavailable                                                          | Item is available when added to the cart                                   | 1: Customer adds 'Spring Rolls' to the cart<br>2: Admin marks 'Spring Rolls' unavailable before checkout<br>3: Customer submits the order             | Cart: Spring Rolls x2, marked unavailable before submission    | Order submission is blocked and the customer is notified that 'Spring Rolls' was removed as unavailable   |               |             |

| Test Case ID | Name of Module                          | Test case description                                                                                             | Pre-conditions                                        | Test Steps                                                                                                                                                                    | Test data                                                         | Expected Results                                                                                                 | Actual Result | Test Result |
|--------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|---------------|-------------|
|              |                                         | after adding it to the cart (REQ-11)                                                                              |                                                       |                                                                                                                                                                               |                                                                   |                                                                                                                  |               |             |
| UT_17        | User Authentication and Role Management | Verify a new customer can register with a valid email/phone and password (REQ-12)                                 | The email/phone used is not already registered        | 1: Submit the registration form with a valid email and password<br>2: Query the Users table for the new account                                                               | Email: customer1@example.com, Password: Cust@1234                 | A new user account is created with role 'Customer' and the password is stored hashed, not in plain text          |               |             |
| UT_18        | User Authentication and Role Management | Verify registration is rejected for a duplicate email or an invalid password format (REQ-12, negative)            | An account with the same email already exists         | 1: Submit the registration form with an email that is already registered<br>2: Observe the system response                                                                    | Email: customer1@example.com (already registered), Password: weak | Registration is rejected with a validation error indicating the email is already in use                          |               |             |
| UT_19        | User Authentication and Role Management | Verify passwords are stored using a secure hash and never in plain text (REQ-12)                                  | A user account has just been created                  | 1: Register a new user<br>2: Inspect the stored password value directly in the database                                                                                       | Email: customer2@example.com, Password: Cust@5678                 | The stored value is a hashed string (e.g., PBKDF2/Argon2 format); the plain-text password is not present         |               |             |
| UT_20        | User Authentication and Role Management | Verify a logged-in user's session/token is invalidated after the configured inactivity period (REQ-14)            | User is logged in and holds a valid session/token     | 1: Log in and note the issued session/token<br>2: Remain inactive for longer than the configured timeout period<br>3: Attempt to access a protected page using the same token | Inactivity period: configured timeout (e.g., 30 minutes)          | The session/token is rejected as expired/invalid and the user is prompted to log in again                        |               |             |
| IT_09        | User Authentication and Role Management | Verify an Admin-role user can access menu-management endpoints while a Customer-role user is denied (REQ-13)      | One Admin account and one Customer account exist      | 1: Log in as Admin and call the menu-management endpoint<br>2: Log in as Customer and call the same endpoint                                                                  | Admin credentials;<br>Customer credentials                        | Admin request succeeds (200 OK);<br>Customer request is denied (403 Forbidden)                                   |               |             |
| IT_10        | User Authentication and Role Management | Verify login issues a valid session/token that is accepted by protected API endpoints (REQ-12, REQ-13)            | A registered user account exists                      | 1: Log in with valid credentials and capture the issued token<br>2: Call a protected endpoint using that token                                                                | Email: customer2@example.com, Password: Cust@5678                 | Login succeeds and returns a valid token; the protected endpoint accepts the token and returns the expected data |               |             |
| ST_07        | User Authentication and Role Management | End-to-end: customer registers, logs in, and books a table using their authenticated session (REQ-12)             | Email used for registration is not already registered | 1: Register a new customer account<br>2: Log in with the new account<br>3: Use the authenticated session to book a table                                                      | Email: customer3@example.com, Password: Cust@9012                 | Registration, login and booking all succeed, with the booking correctly associated with the new account          |               |             |
| ST_08        | User Authentication and Role Management | End-to-end: a Staff-role user attempts to access an Admin-only menu-management page and is denied access (REQ-13) | A Staff account exists and is logged in               | 1: Log in as Staff<br>2: Navigate directly to the Admin-only menu-management page                                                                                             | Staff credentials                                                 | Access is denied with an appropriate message; the menu-management page is not displayed                          |               |             |

- **Test Case ID** : Each test case should be represented by a unique ID. To indicate test types, follow some convention like "UT\_01" indicating "Unit Testing - Test Case#1."
- **Name of the module** : Specify the name of the **main module or sub module** being tested
- **Test Case Description** : Specify the summary or test purpose in brief
- **Pre- Conditions** : Any requirement that needs to be done before execution of this test case.
- **Test Steps** : Mention all the steps in detail and specify the order in which it is to be executed.
- **Test Data** : Input for the test case to be executed. Specify different data sets with precise values to be used as input. (create test case for both valid and invalid inputs)
- **Expected Results** : Mention the expected results including error or precise messages that should be displayed on screen
- **Actual Results** : After execution of test case fill this column with the result obtained
- **Test Result (Pass/Fail)** : Mark this field as “fail” if the actual result is not same as expected result else mark as “pass”.

### Sample Test Case:

| Test Case ID | Name of Module           | Test case description           | Pre-conditions           | Test Steps                                                                                                                | Test data                                         | Expected Results                                  | Actual Result                                     | Test Result |
|--------------|--------------------------|---------------------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|---------------------------------------------------|---------------------------------------------------|-------------|
| UT-01        | User registration module | To test the login functionality | Access to Chrome Browser | 1: Navigate to <a href="http://www.demo.com">http://www.demo.com</a><br>2: Enter Username and Password<br>3: Click Submit | User name: PESU Student.<br><br>Password : pes123 | Login should be successful with “welcome message” | Login successful with “welcome message” displayed | Pass        |
|              |                          |                                 |                          |                                                                                                                           |                                                   |                                                   |                                                   |             |